

Lighten_VLA_离线推理运行文档

Lighten_VLA：从环境激活到 Alpamayo 1.5 离线推理测试

项目根目录： `/home/tuqiang/lang/Lighten_VLA`

当前验证平台：Ubuntu 22.04 / x86_64 / NVIDIA L20 / CUDA 可用

当前 Alpamayo 运行环境：Python 3.12.14、PyTorch 2.8.0+cu128、FlashAttention 2.8.3、Transformers 4.57.1、physical-ai-av 0.2.0

1. 项目目标

本项目当前阶段的目标是：

1. 在 L20 服务器上完整加载 NVIDIA Alpamayo 1.5 10B 原始 checkpoint；
2. 在完全离线环境下完成固定测试 clip 的端到端推理；
3. 保留可复现的输入、随机种子、模型版本、环境、输出、精度和性能记录；
4. 以当前 BF16 原始模型结果作为后续剪枝、蒸馏、量化和视觉 Token 剪枝的基准线。

当前已经完成：

- L20 环境验收；
- 官方源码和 checkpoint 校验；
- Python/CUDA 环境离线重建；
- 离线资产 `preflight-only` 检查；
- Alpamayo 1.5 10B 离线兼容性端到端推理；
- L20 性能基准测试。

2. 项目目录结构

当前服务器项目根目录：

```
/home/tuqiang/lang/Lighten_VLA/
```

主要结构如下：

```
Lighten_VLA/
├── alpamayo1.5/
│   ├── src/
│   │   ├── alpamayo1_5/
│   │   │   ├── test_inference.py
│   │   │   ├── helper.py
│   │   │   ├── load_physical_aiavdataset.py
│   │   │   ├── config.py
│   │   │   ├── models/
│   │   │   ├── diffusion/
│   │   │   └── ...
│   ├── pyproject.toml
│   ├── uv.lock
│   └── .venv/
├── models/
│   ├── config.json
│   ├── model.safetensors.index.json
│   ├── model-00001-of-00005.safetensors
│   ├── model-00002-of-00005.safetensors
│   ├── model-00003-of-00005.safetensors
│   ├── model-00004-of-00005.safetensors
│   ├── model-00005-of-00005.safetensors
│   ├── LICENSE
│   └── README.md
├── alp15-test-clip-delivery-20260917/
│   ├── dataset-manifest.json
│   └── datasets--nvidia--PhysicalAI-Autonomous-Vehicles/
│       ├── blobs/
│       └── snapshots/
├── alp15-configs-delivery-20260919/
│   ├── dependencies/
│   │   └── configs/
│   │       ├── Cosmos-Reason2-8B/
│   │       │   ├── config.json
│   │       │   ├── generation_config.json
│   │       │   ├── preprocessor_config.json
│   │       │   ├── video_preprocessor_config.json
│   │       │   ├── tokenizer.json
│   │       │   ├── tokenizer_config.json
│   │       │   ├── chat_template.json
│   │       │   ├── merges.txt
│   │       │   └── vocab.json
│   │       └── Qwen3-VL-2B-Instruct/
│   │           ├── config.json
│   │           └── generation_config.json
```

```

├── ── processor_config.json
├── ── video_preprocessor_config.json
├── ── tokenizer.json
├── ── tokenizer_config.json
├── ── chat_template.json
├── ── merges.txt
├── ── vocab.json
├── manifests/
│   ├── cosmos-reason2-source.json
│   └── qwen3-vl-source.json
├── alpamayo15-linux-amd64-uv-20260919/
│   ├── bin/
│   │   └── uv
│   ├── uv-python/
│   │   └── cpython-3.12.14-linux-x86_64-gnu/
│   ├── uv-cache/
│   ├── verify-venv/
│   ├── manifests/
│   │   ├── validation.json
│   │   ├── uv.lock
│   │   └── pyproject.toml
│   ├── logs/
│   └── README.md
├── scripts/
│   ├── audit_l20_readiness.sh
│   ├── test_alpamayo15_offline_compat.py
│   ├── benchmark_alpamayo15_offline.py
│   ├── prepare_alpamayo15_offline_bundle.sh
│   ├── verify_alpamayo15_offline_bundle.sh
│   └── ...
├── docs/
├── tests/
├── logs/
│   ├── offline_compat_e2e_20260920.log
│   ├── alp15_l20_baseline_console_20260920.log
│   └── ...
└── AGENTS.md

```

说明：

- `alpamayo1.5/`：NVIDIA 官方 Alpamayo 1.5 源码。
- `models/`：Alpamayo 1.5 10B checkpoint。
- `alp15-test-clip-delivery-*`：固定测试 clip 的离线数据。
- `alp15-configs-delivery-*`：Cosmos 与 Qwen 的非权重配置。
- `alpamayo15-linux-amd64-uv-*`：Linux/x86_64 离线 Python、uv 和依赖缓存。

- `scripts/`：项目验收、离线兼容推理和基准测试脚本。
- `logs/`：推理和 benchmark 日志。
- `.venv/`：在 L20 上通过离线 `uv sync` 原生重建出的正式环境。

3. 第一次部署时：重建离线环境

如果 `.venv` 已经存在并且已验证通过，可以跳过本节，直接进入第 4 节。

进入项目根目录：

```
cd /home/tuqiang/lang/Lighten_VLA
```

设置依赖和源码路径：

```
export ALP15_DEPS=/home/tuqiang/lang/Lighten_VLA/alpamayo15-  
linux-amd64-uv-20260919  
export ALP15_SOURCE=/home/tuqiang/lang/Lighten_VLA/alpamayo  
1.5  
  
export UV_CACHE_DIR="$ALP15_DEPS/uv-cache"  
export UV_PYTHON_INSTALL_DIR="$ALP15_DEPS/uv-python"  
export UV_PROJECT_ENVIRONMENT="$ALP15_SOURCE/.venv"
```

检查官方 `uv.lock`：

```
sha256sum "$ALP15_SOURCE/uv.lock"
```

期望 SHA256：

```
6abf4ccba882e0b7849880ccaef794cbeecc36a43a2a12007e6b30632ae4e8b1
```

完全离线重建环境：

```
"$ALP15_DEPS/bin/uv" sync \  
--project "$ALP15_SOURCE" \  
--no-build-isolation
```

```
--frozen \  
--offline
```

参数含义：

```
--project    指定 Alpamayo 官方项目  
--frozen     严格使用冻结的 uv.lock，不重新解析依赖  
--offline    禁止联网
```

4. 激活运行环境

每次登录服务器后，先：

```
cd /home/tuqiang/lang/Lighten_VLA
```

激活 Alpamayo 环境：

```
source alpamayo1.5/.venv/bin/activate
```

确认 Python：

```
which python  
python --version
```

预期：

```
/home/tuqiang/lang/Lighten_VLA/alpamayo1.5/.venv/bin/python  
Python 3.12.14
```

确认关键依赖：

```
python - <<'PY'  
import torch  
import flash_attn  
import physical_ai_av  
import transformers
```

```

from alpamayo1_5.models.alpamayo1_5 import Alpamayo1_5

print("torch:", torch.__version__)
print("torch CUDA build:", torch.version.cuda)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("BF16:", torch.cuda.is_bf16_supported())

print("flash_attn:", flash_attn.__version__)
print("physical_ai_av:", physical_ai_av.__name__)
print("transformers:", transformers.__version__)
print("Alpamayo:", Alpamayo1_5.__name__)
PY

```

当前已验证结果：

```

torch: 2.8.0+cu128
torch CUDA build: 12.8
CUDA available: True
GPU: NVIDIA L20
BF16: True
flash_attn: 2.8.3
transformers: 4.57.1
Alpamayo: Alpamayo1_5

```

5. 服务器和模型资产快速检查

如需重新做服务器审计：

```

cd /home/tuqiang/lang/Lighten_VLA

AUDIT_DIR="$PWD/alp15_l20_audit_$(date +%Y%m%d_%H%M%S)"

CODE_DIR="$PWD/alpamayo1.5" \
MODEL_DIR="$PWD/models" \

```

```
DATA_DIR="$PWD/alp15-test-clip-delivery-20260917" \  
bash scripts/audit_l20_readiness.sh "$AUDIT_DIR"
```

查看结果：

```
cat "$AUDIT_DIR/readiness.log"
```

当前已经确认：

```
系统: Ubuntu 22.04 / x86_64  
GPU: NVIDIA L20  
显存: 约 46 GB  
Compute Capability: 8.9  
BF16: 支持  
checkpoint 五个分片: SHA256 全部 MATCH  
官方源码 HEAD:  
36aeb4c5938cbc2eb2aed33b22434773da4ab639
```

6. 推理前离线资产预检

正式加载模型前，先执行 `--preflight-only`。

```
cd /home/tuqiang/lang/Lighten_VLA
```

执行：

```
python -B scripts/test_alpamayo15_offline_compat.py \  
  --model-dir /home/tuqiang/lang/Lighten_VLA/models \  
  --dataset-dir /home/tuqiang/lang/Lighten_VLA/alp15-test-clip-delivery-20260917 \  
  --cosmos-config-dir /home/tuqiang/lang/Lighten_VLA/alp15-configs-delivery-20260919/dependencies/configs/Cosmos-Reason2-8B \  
  --qwen-config-dir /home/tuqiang/lang/Lighten_VLA/alp15-configs-delivery-20260919/dependencies/configs/Qwen3-VL-2B-Instruct \  
  --config-manifest-dir /home/tuqiang/lang/Lighten_VLA/alp1
```

```
5-configs-delivery-20260919/manifests \
--preflight-only
```

这一步检查：

- Alpamayo checkpoint 文件；
- checkpoint SHA256；
- checkpoint index；
- 固定 PhysicalAI AV clip；
- dataset manifest；
- Cosmos 9 个非权重配置；
- Qwen 9 个非权重配置；
- 两个配置来源 revision；
- `physical-ai-av==0.2.0`；
- 固定 `clip_id`、`t0_us` 和 `chunk_id`；
- Python 外联是否被阻止。

正常结果应包含：

```
ALP15 离线兼容性测试：本地资产预检通过
clip_id=030c760c-ae38-49aa-9ad8-f5650a545d26; t0_us=5100000; chunk_id=3119
```

7. 正式离线端到端推理

预检通过后，执行完整推理。

建议保存日志：

```
mkdir -p logs
```

执行：

```
set -o pipefail

python -B scripts/test_alpamayo15_offline_compat.py \
```

```

--model-dir /home/tuqiang/lang/Lighten_VLA/models \
--dataset-dir /home/tuqiang/lang/Lighten_VLA/alp15-test-clip-delivery-20260917 \
--cosmos-config-dir /home/tuqiang/lang/Lighten_VLA/alp15-configs-delivery-20260919/dependencies/configs/Cosmos-Reason2-8B \
--qwen-config-dir /home/tuqiang/lang/Lighten_VLA/alp15-configs-delivery-20260919/dependencies/configs/Qwen3-VL-2B-Instruct \
--config-manifest-dir /home/tuqiang/lang/Lighten_VLA/alp15-configs-delivery-20260919/manifests \
2>&1 | tee logs/offline_compat_e2e_$(date +%Y%m%d_%H%M%S).log

echo "exit_code=${PIPESTATUS[0]}"

```

当前固定推理参数：

```

clip_id = 030c760c-ae38-49aa-9ad8-f5650a545d26
t0_us = 5100000
chunk_id = 3119

top_p = 0.98
temperature = 0.6
num_traj_samples = 1
max_generation_length = 256
seed = 42
dtype = BF16

```

当前已验证成功输出：

```

Chain-of-Causation:
Nudge to the left to clear the construction equipment blocking the right side of our lane

minADE:
0.371870219707489 meters

pred_xyz shape:
(1, 1, 1, 64, 3)

pred_rot shape:
(1, 1, 1, 64, 3, 3)

```

```
exit_code=0
```

含义：

```
VLM
↓
生成驾驶场景推理 CoC
↓
Diffusion Expert
↓
生成 64 个未来轨迹点
↓
与 GT 轨迹比较
↓
得到 minADE
```

8. 推理时观察 GPU

可以另外开一个 SSH 终端：

```
watch -n 1 nvidia-smi
```

或者记录 GPU 状态：

```
nvidia-smi \
  --query-gpu=timestamp,memory.used,memory.free,utilization.gpu,power.draw \
  --format=csv \
  -l 1 \
  > logs/gpu_monitor_$(date +%Y%m%d_%H%M%S).csv
```

结束推理后：

```
Ctrl + C
```

停止监控。

9. 正式性能基准测试

当前基准测试脚本：

```
scripts/benchmark_alpamayo15_offline.py
```

它测量：

```
模型加载  
数据读取  
Processor  
Host → GPU  
Cold inference  
Steady-state inference  
VLM generate  
Diffusion sample  
GPU 显存  
minADE  
CoC  
重复运行稳定性
```

执行：

```
cd /home/tuqiang/lang/Lighten_VLA  
  
set -o pipefail  
  
python -B scripts/benchmark_alpamayo15_offline.py \  
  --source-dir /home/tuqiang/lang/Lighten_VLA/alpamayo1.5 \  
  --model-dir /home/tuqiang/lang/Lighten_VLA/models \  
  --dataset-dir /home/tuqiang/lang/Lighten_VLA/alp15-test-c  
lip-delivery-20260917 \  
  --cosmos-config-dir /home/tuqiang/lang/Lighten_VLA/alp15-  
configs-delivery-20260919/dependencies/configs/Cosmos-Reaso  
n2-8B \  
  --qwen-config-dir /home/tuqiang/lang/Lighten_VLA/alp15-co  
nfigs-delivery-20260919/dependencies/configs/Qwen3-VL-2B-In  
struct \  
  --config-manifest-dir /home/tuqiang/lang/Lighten_VLA/alp1  
5-configs-delivery-20260919/manifests \  
  --warmup-runs 1 \  
  --
```

```
--runs 5 \
2>&1 | tee logs/alp15_l20_baseline_console_$(date +%Y%m%d
%H%M%S).log

echo "benchmark_exit_code=${PIPESTATUS[0]}"
```

当前基准测试结果：

总参数量	11.0785B
VLM	8.7979B
Expert	2.2792B
BF16 参数存储	20.64 GiB
Cold inference	2384.64 ms
Steady mean	1615.68 ms
Steady median	1614.31 ms
Steady P95	1619.31 ms
Steady stdev	2.74 ms
VLM generate	1364.01 ms
Diffusion sample	252.78 ms
Peak allocated VRAM	22126 MiB
Peak reserved VRAM	22502 MiB
minADE	0.371870 m

阶段耗时大致为：

```
完整推理: 约 1621 ms
├── VLM generate: 约 1364 ms
│   └── ≈ 84%
├── Diffusion sample: 约 253 ms
│   └── ≈ 16%
└── 其他: 极少
```

因此当前主要计算瓶颈位于 VLM。

10. 当前输入规模

本次固定测试输入：

```
4 个摄像头
×
每个摄像头 4 帧
=
16 张图像
```

原始 frame tensor：

```
(4, 4, 3, 1080, 1920)
```

即：

```
camera = 4
frame per camera = 4
RGB = 3
height = 1080
width = 1920
```

Processor 后：

```
input_ids shape = (1, 3086)
input_token_count = 3086
```

11. 当前模型规模

实际 checkpoint 加载后的参数统计：

```
Alpamayo 1.5
├── VLM
│   └── 8.7979B
├── Diffusion Expert
│   └── 2.2792B
├── action_in_proj
│   └── 1.35M
└── action_out_proj
    └── 4K
```

Total:
11.0785B

当前 Attention:

VLM:
FlashAttention 2

Expert:
PyTorch SDPA

12. 当前显存情况

BF16 模型参数理论存储:

20.64 GiB

稳定推理时:

PyTorch peak allocated:
约 21.61 GiB

PyTorch peak reserved:
约 21.97 GiB

GPU device used:
约 22.4 GiB~22.9 GiB

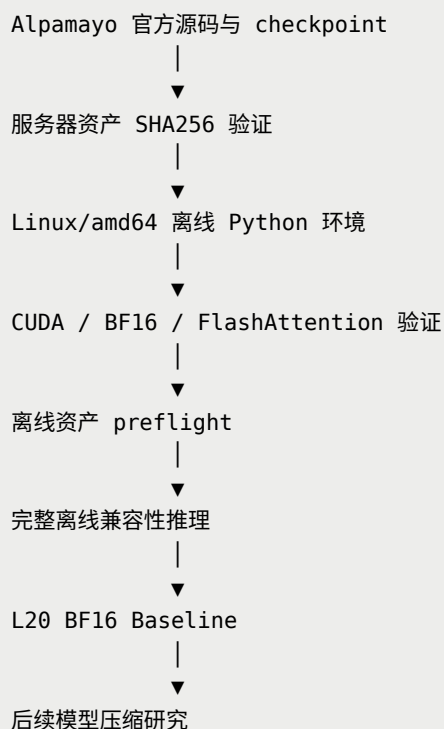
因此原始 BF16 Alpamayo 1.5 无法直接部署到 16GB Jetson Orin NX。

后续必须依赖:

模型压缩
+
量化
+
Token 剪枝
+
部署级 Runtime 优化

13. 当前项目状态

当前工作流：



当前已经完成：

- [✓] 官方源码冻结
- [✓] checkpoint 验证
- [✓] 测试 clip 验证
- [✓] Cosmos 配置
- [✓] Qwen Processor 配置
- [✓] Linux/amd64 离线依赖
- [✓] L20 CUDA 环境
- [✓] 离线 preflight
- [✓] 10B checkpoint 加载
- [✓] CoC 生成
- [✓] Diffusion 轨迹生成
- [✓] minADE 计算
- [✓] 性能 benchmark

14. 日常最快运行流程

如果服务器已经配置完成，正常情况下以后只需要下面几步。

14.1 激活环境

```
cd /home/tuqiang/lang/Lighten_VLA
source alpamayo1.5/.venv/bin/activate
```

14.2 确认 GPU

```
python - <<'PY'
import torch
print(torch.__version__)
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
PY
```

应得到类似：

```
2.8.0+cu128
True
NVIDIA L20
```

14.3 可选：先跑预检

```
python -B scripts/test_alpamayo15_offline_compat.py \
  --model-dir "$PWD/models" \
  --dataset-dir "$PWD/alp15-test-clip-delivery-20260917" \
  --cosmos-config-dir "$PWD/alp15-configs-delivery-20260919/dependencies/configs/Cosmos-Reason2-8B" \
  --qwen-config-dir "$PWD/alp15-configs-delivery-20260919/dependencies/configs/Qwen3-VL-2B-Instruct" \
  --config-manifest-dir "$PWD/alp15-configs-delivery-20260919/manifests" \
  --preflight-only
```

14.4 正式推理

```
python -B scripts/test_alpamayo15_offline_compat.py \
  --model-dir "$PWD/models" \
  --dataset-dir "$PWD/alp15-test-clip-delivery-20260917" \
  --cosmos-config-dir "$PWD/alp15-configs-delivery-20260919/dependencies/configs/Cosmos-Reason2-8B" \
  --qwen-config-dir "$PWD/alp15-configs-delivery-20260919/dependencies/configs/Qwen3-VL-2B-Instruct" \
  --config-manifest-dir "$PWD/alp15-configs-delivery-20260919/manifests"
```

这就是当前服务器上最简洁、可复现的 Alpamayo 1.5 离线推理流程。

15. 重要命名边界

当前成功的是：

Alpamayo 1.5 10B
L20 完全离线兼容性端到端推理成功

不要写成：

NVIDIA 官方原始入口完全离线跑通

因为当前正式成功链路使用的是：

scripts/test_alpamayo15_offline_compat.py

它对模型路径、Qwen/Cosmos 配置位置以及 `physical_ai_av` 的离线行为做了最小兼容适配。

该适配不改变：

模型权重
模型结构
固定 clip
VLM 推理
Diffusion Expert

采样参数
minADE 计算

但在项目记录和实验报告中，仍应将其准确称为：

离线兼容性测试 / 离线兼容性基线

16. 后续实验基准要求

从现在开始，后续剪枝、量化、视觉 Token 剪枝或 Runtime 优化都建议保持以下条件不变：

同一个 clip
同一个 t0_us
同一个 seed
同一个 top_p
同一个 temperature
同一个 num_traj_samples
同一个 max_generation_length
同一种计时方法
同一种 minADE 计算方法

以后每个优化版本至少比较：

Parameters
Peak GPU Memory
Steady-state Latency
VLM Latency
Diffusion Latency
Input Token Count
minADE
CoC / 输出合理性

最终形成：

模型版本	Params	Peak VRAM	Latency	minADE
BF16 Baseline	11.08B	22.1 GiB	1615.7 ms	0.3719 m
剪枝	待测	待测	待测	待测
剪枝 + 量化	待测	待测	待测	待测

模型版本	Params	Peak VRAM	Latency	minADE
+ Token Pruning	待测	待测	待测	待测

这张表将作为后续整个边缘部署优化阶段的主要实验对照。